

exp-fips203-mlkem-kem-decaps-ct-k2

FIPS 203 Alg.21 ML-KEM-512 KEM Decaps 实现方案

ascendc 工程 (ML-KEM-512 W4b / E21ct)

2026-07-27

摘要

本文档描述 `examples/incubating/ml-kem/ml-kem-512/exp-fips203-mlkem-kem-decaps-ct-k2/`: 在 **Atlas A2 / Ascend910B4** 上以 AscendC 实现 FIPS 203 **Algorithm 21**, 即 ML-KEM-512 ($k=2$) **KEM Decaps CT** 专题副本。

参数锁定: 参数卡与 `ascendc-tests/ml-kem/ml-kem-512/pass-fix-f203-alg21-kem-decaps-device-ct-k2/STATUS.md`: `dk_kem=1632B`, `c=768B`, 输出 `K=32B`; 拒绝路径必须输出 $J(z||c)$ 且 `reject` $\neq$ `accept`; SIM/生产默认 `decaps_2session`, T19i 形态 **3 launch**。**组成**: Phase-D 复用 E15/D15 k2 Decrypt fused; Phase-E 复用 E14/D14 k2 Encrypt 几何, 并在设备侧完成 $G(m'||h)$ 、再加密、密文比较与 FO 选路。**计划 AI Core 数**: Phase-D 为 MIX `blockDim=1`; Phase-E prep 为 AIV_ONLY `blockDim=2`; Phase-E compute+pack+FO 为 MIX `blockDim=1` (S-1)。**禁止**: 写入 `stable-512`; 从 `frozen/` 抄码; 沿用 k3/k4 字节长或零垫。

目录

1	工程边界	2
1.1	禁止项	2
2	数学语义 (Alg.21 $\rightarrow$ Alg.18)	2
3	AscendC 实现规划	3
3.1	GM 几何	3
3.2	同步与流水	4
4	AscendC API 表	4
5	数学步骤覆盖率	5
6	验证计划	5
7	产出物	6

1 工程边界

来源	用途
ascendc-tests/ml-kem/ml-kem-512/pass-fix-f203-alg21-kem-decaps-device-ct-k2/	活跃 E21ct k2 行为基线；可复制后自包含适配
examples/incubating/ml-kem/ml-kem-512/exp-fips203-mlkem-pke-decrypt-k2/	活跃 E15 incubating PKE Decrypt
examples/incubating/ml-kem/ml-kem-512/exp-fips203-mlkem-pke-encrypt-k2/	活跃 E14 incubating PKE Encrypt
examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-kem-decaps-k3/	活跃 k3 incubating 结构参考；仅目录组织与状态口径
library/shared/CANN / tikicpulib / CAModel	SHA3/SHAKE、统一整数压缩等编译与运行环境

1.1 禁止项

- 禁止从 frozen/ 拿源码、customspec 或路线。
- 禁止把 m' 、 K' 、 r' 、 c' 、 h 、 z 等中间态作为默认 I/O 落盘。
- 禁止 Host 预填 r' 、Host memcmp 或 Host SHAKE256 选 K 冒充生产 FO。
- 禁止为 G 或 FO 新增与 Encrypt/Decrypt 无关的产品化独立 launch；默认路径须保持 T19i 三段结构。
- 禁止将本 E21ct CT 默认改回 decaps_1session；1-session 仅作为排障覆盖。

2 数学语义 (Alg.21 $\rightarrow$ Alg.18)

张量 / 文件	契约
input/dk_kem.bin	1632B, 展开为 $dk_{PKE}(768) ek(800) h(32) z(32)$
input/c.bin	768B, $c = c_1 c_2$ (640+128), $d_u = 10, d_v = 4$

input/lut_even_stack.bin	Decrypt/Encrypt NTT/INTT Stage2 LUT
lut_odd_stacked.bin	
output/K.bin	32B; 合法路径为 K' , 拒绝路径为 $J(z c)$, 且必须与合法路径 K 不同

$$\begin{aligned}
m' &\leftarrow \text{K-PKE.Decrypt}(\text{dk}_{\text{PKE}}, c) \\
(K' || r') &\leftarrow G(m' || h) = \text{SHA3-512}(m' || h) \\
c' &\leftarrow \text{K-PKE.Encrypt}(\text{ek}, m', r') \\
K &\leftarrow \begin{cases} K' & \text{若 } c = c' \\ J(z || c) = \text{SHAKE256}(z || c, 32) & \text{若 } c \neq c' \end{cases}
\end{aligned}$$

行 1-4 只做 `dk_kem` 指针切片, 不另开 launch; 行 8-12 的比较与 J 选路在设备 FO 中完成。

3 AscendC 实现规划

Launch	核类型 / blockDim	数据划分与理由
Phase-D	MIX, blockDim=1	继承 E15/D15 k2 fused; 输出 m' 到 GM workspace。
Phase-E prep	AIV_ONLY, blockDim=2	block0 先执行 $G(m' h)$; 随后继承 E14/D14 prep: $\hat{A}[4] \ 2+2$, <code>re[5]</code> 由 r' 采样。
Phase-E compute+FO	MIX, blockDim=1	复用 E14/D14 compute; 本地 118_119 尾 pack 出 c' 后同核 <code>KemDecFo</code> 写 K.bin。

3.1 GM 几何

对象	锁定几何
<code>dk_kem</code>	1632B; 偏移: <code>dk_pke[0,768)</code> , <code>ek[768,1568)</code> , <code>h[1568,1600)</code> , <code>z[1600,1632)</code>
<code>c / c'</code>	768B; $d_u = 10, d_v = 4$, <code>c1=640B</code> , <code>c2=128B</code>
$\hat{A}$	[4,256] int32; 禁止 pad
<code>re</code>	[5,256] int32; 随机性来自设备 r'
Phase-E INTT	真 polyvec4: <code>S0 [8,256] int8</code> , <code>mat_c [32,128] int32</code>
<code>Kprime / Kout</code>	各 32B; <code>Kout</code> 为唯一 D2H 输出

3.2 同步与流水

Phase-D 与 Phase-E 之间由 Host launch 顺序保证 GM 可见性;SIM 默认保持 decaps_2session, 即 Phase-D 与 Phase-E 分 session 执行。Phase-E prep 内 G 写出 $K'\|r'$ 后, PRF/CBD 必须读取完整 r' 。Phase-E compute 内部沿用 E14/D14 同步; AIV 双分片 pack 完成后使用 SyncAll 汇合, 再由固定 AIV 执行 FO。CPU 仅作为逻辑孪生; SIM 是同步与搬运验收门槛。

4 AscendC API 表

本实现不引入 E21ct 独有的新 AscendC 矢量/矩阵 API;KEM 头尾使用 shared Keccak Sha3OneShot/Shake2

API 名	分类 / 文档	Atlas A2	本用例 dtype	备注
GetBlockIdx / GetSubBlockIdx	系统变量 / 资源	√	标量索引	区分 AIV 分片与 MIX 子核; KEM 头尾仅固定 block 执行
GlobalTensor / LocalTensor	基础结构	√	uint8/int8/int32	32B/UB 张量契约须与本 spec 几何一致
DataCopy	Memory 数据搬运	√	uint8/int8/int32	连续块搬运; 32B 对齐约束
Duplicate	Memory 矢量计算	√	uint8/int32	UB 清零、pad 与 tail buffer 初始化
PipeBarrier	Pipe 同步	√	—	MTE/Vector/Cube 阶段交界显式同步
CrossCoreSetFlag / CrossCoreWaitFlag	MIX 同步	√	—	AIV/AIC 通过 GM 交接; CPU 过不能删
SyncAll	硬同步	√	—	双 AIV pack 完成后汇合, 再由 AIV0 执行 FO
Mmad / Fixpipe	矩阵计算	√	int8*int8->int32	INTT/INTT Stage2 MMAD; k2 真 batch 几何
ShiftRight	矢量算术	√	int32	limb 拆分、Barrett 与 ByteEncode 辅助右移
Muls / Add / Sub	矢量算术	√	int32/int16	limb、模加减、压缩/编码辅助

Cast	类型转换	√ (受限)	int32/int16/half/int8	不得假设 int32->int8 单跳
Gather	矢量搬运/选择	√	int32	仅允许非 NTT S1-S3 段; NTT 三段内禁用
GetValue / SetValue	LocalTensor 标量访问	√	uint8/int32	仅用于 pack 与 KEM 头尾标量缺口

5 数学步骤覆盖率

数学步骤	实现方式 / API	覆盖	缺口说明
切分 dk_kem	Host 传整段 GM, kernel 指针偏移	100% 设备	不另开 launch
$m' = \text{Decrypt}$	E15/D15 fused	100% 设备	真 k2、du/dv=10/4
$G(m' h)$	shared Keccak SHA3-512	100% 设备	K' 与 r' 均在设备 workspace
$\text{Encrypt}(ek, m', r')$	E14/D14 prep+compute	100% 设备	$\hat{A}[4]$ 、re[5]、INTT batch4; 无零垫
$c \stackrel{?}{=} c'$ 与 FO	本地 KemDecFo + SHAKE256 J	100% 设备	SIM 嵌入 118_119 尾

6 验证计划

```
cd examples/incubating/ml-kem/ml-kem-512/exp-fips203-mlkem-kem-decaps-ct-k2
bash run.sh -r cpu -v Ascend910B4
SIM_DIRECT=1 bash run.sh -r sim -v Ascend910B4
```

期望: K.bin 为 32B, 合法路径与 golden 对拍 PASS; SIM 默认 decaps_2session; 用例根无 stray dump。

拒绝路径必验:

```
KEM_DECAPS_REJECT=1 bash run.sh -r cpu -v Ascend910B4
KEM_DECAPS_REJECT=1 SIM_DIRECT=1 bash run.sh -r sim -v Ascend910B4
```

7 产出物

- `exp-fips203-mlkem-kem-decaps-ct-k2-实现方案-customspec.tex`
- `exp-fips203-mlkem-kem-decaps-ct-k2-实现方案-customspec.pdf`
- 后续实现文件须与本 `customspec` 同目录自包含，且自研 Python/C/AscendC 写详细中文注释。